Interface Description

Interface Description 1
I、 Captute Interface Description 3
1. Initialize the collector 3
2. Release the collector 3
3. Image acquisition in synchronous mode 3
4. Collect one frame of image 3
5. Set Image Type 3
6. Obtain a specified size scrolling fingerprint image 3
7. Obtain single finger images 3
8. Beep control 4
9. Set screen image display 4
10. Set LED display 4
11. Image acquisition interface error code 4
12. Image code 5
II、 Libfingeralgo.so Interface Description 6
1. Initialize 6
2. Release 6
3. Splitting fingerprints 6
Return:Call successful, return finger nums 6
4. Initialize the splicing dynamic library 6
5. Release the splicing dynamic library 6
6. Does the splicing interface provide a function to determine whether an image is a fingerprint 6
7. Does the splicing interface provide a function for judging image quality 6
8. Does the splicing interface provide a function to determine the quality of fingerprints 6
9. Does the interface provide image enhancement function for stitching fingerprints . 7
10. Determine whether the rolling collection function is supported 7
11. Choose splicing method 7
12. Initialize the splicing process 7
13. Splicing process 7
14. End splicing 8
15. Judging image quality 8
16. Judging fingerprint quality 8
17. Enhance images 8
18. Determine if the image is a fingerprint 8
19. Obtain splicing interface error information 9
20. Obtain the version of the interface 9
21. Instructions for obtaining splicing interfaces 9

22. Image stitching interface error code 9
III、 libFpStdAPI.so Interface Description 11
1. Initialization algorithm 11
2. Release algorithm 11
3. Get Image Quality 11
4. Get Image NFIQ Quality 11
5. Comparing fingerprint features 11
6. Generate fingerprint ANSI features 11
7. Generate fingerprint ISO features 12
8. CompressToWSQImage 12
9. UnCompressFromWSQImage 12

Ⅰ、Captute Interface Description

1. Initialize the collector

Function prototype :public int LIVESCAN_Init(boolean isLcdScreenMode)
Parameter :boolean isLcdScreenMode; Is it in LCD screen mode?
Return: **Initialize** successful, return 1

2. Release the collector

Function prototype :public int LIVESCAN_Close();
Parameter :null
Return: **Release** successful, return 1

3. Image acquisition in synchronous mode

Function prototype :public int captureFinger(int timeout , byte [] capturedFinger , byte [] captureBmp);
Parameter :int timeout eg：100
byte [] capturedFinger : Point to the memory block that stores the collected data.OutParameter .
byte [] captureBmp BMP capture；null
Return:Call successful, return 1

4. Collect one frame of image

Function prototype :public int LIVESCAN_GetFPRawData(int nChannel, byte [] pRawData);
Parameter :int nChannel channel number.inputParameter
byte [] pRawData Point to the memory block that stores the collected data.OutParameter .
Return:Call successful, return 1

5. Set Image Type

Function prototype :public int LIVESCAN_SetFingerDryWet(int nLevel);
Parameter :int nLevel 3:wet，4:normal，5:dry.inputParameter
Return:Call successful, return 0

6. Obtain a specified size scrolling fingerprint image

Function prototype:public int LIVESCAN_GetRollFPRawData(byte [] pRawData , int nWidth , int nHeight)
Parameter:byte [] pRawData Point to the memory block that stores the collected data. Output parameter.
int width image width.inputParameter.
int height image height.inputParameter.
Return:Call successful, return 1

7. Obtain single finger images

Function prototype:public int LIVESCAN_GetFlatFPRawData(byte [] pRawData , int nWidth , int nHeight)
Parameter:byte [] pRawData Point to the memory block that stores the collected da ta. Output parameter.
int width image width.inputParameter.
int height image height.inputParameter.
Return:Call successful, return 1

8. Beep control

Function prototype:public int LIVESCAN_Beep(int beepType);
Parameter:int beepType Beeping frequency.inputParameter.
Return:Call successful, return 1

9. Set screen image display

Function prototype:public void LcdImage(int index)
Parameter:int Index Image code,Reference Image Code Table.inputParameter.
Return:Call successful, return 1

10. Set LED display

Function prototype:public void LedImage(int index)
Parameter:int Index Image code,Reference Image Code Table.inputParameter.
Return:Call successful, return 1

11. Image acquisition interface error code

The error codes for the image acquisition interface refer to Table 1.1.

Table 1.1 Error code description

Error code	
-1	Parameter error. There is an error in the parameters of the given function
-2	Memory allocation failed. Not enough memory allocated
-3	Function not implemented. The function calling function has not been implemented
-4	The device does not exist. During initialization, it was detected that the device does not exist
-5	Device not initialized
-6	Illegal error number
-7	No authorization
-8	Denied the permission request
-10	Time out

12. Image code

Code	
2	Collect the four fingers of the left hand
3	Collect the four fingers of the right hand
4	Collect thumbs from both hands
5	Collect the left thumb
6	Collect left index finger
7	Collect left middle finger
8	Collect left ring finger
9	Collect left and right little fingers
10	Collect right thumb
11	Collect right index finger
12	Collect right middle finger
13	Collect the right ring finger
14	Collect the right little finger
15	Successfully collected the four fingers of the left hand
16	Failed to collect the four fingers of the left hand
17	Successfully collected the four fingers of the right hand
18	Failed to collect the four fingers of the right hand
19	Successfully collected thumbs from both hands
20	Failed to collect thumbs from both hands
21	Fingerprint collection successful
22	Fingerprint collection failed

II、Libfingeralgo.so Interface Description

1. Initialize

Function prototype:public static native int Init(byte [] distortionData)

2. Release

Function prototype:public static native int Release()

3. Splitting fingerprints

Function prototype :public static int DoSplit(byte [] pImgBuf, int nImgW, int nImgH, int nSplitW, int nSplitH, FPSPLIT_INFO[] pInfo)
Parameter ： byte [] pImgBuf image data.inputParameter .
int nImgW image width.inputParameter .
int nImgH image height.inputParameter .
int nSplitW Split image width.inputParameter .
int nSplitH The height of the split image.inputParameter .
struct FPSPLIT_INFO_ pInfo[] Split fingerprint image data.OutParameter .

Return:Call successful, return finger nums

4. Initialize the splicing dynamic library

Function prototype :public static native int MOSAIC_Init()
Parameter ： null.
Return:Call successful, return 1

5. Release the splicing dynamic library

Function prototype :public static native int MOSAIC_Close()
Parameter :null.
Return:Call successful, return 1

6. Does the splicing interface provide a function to determine whether an image is a fingerprint

Function prototype :public static native int MOSAIC_IsSupportIdentifyFinger()
Parameter :null.
Return:If it supports determining whether the image is a fingerprint, return 1

7. Does the splicing interface provide a function for judging image quality

Function prototype :public static native int MOSAIC_IsSupportImageQuality()
Parameter :null.
Return:If image quality judgment is supported, return 1

8. Does the splicing interface provide a function to determine the quality of fingerprints

Function prototype :public static native int MOSAIC_IsSupportFingerQuality()

Parameter ： null.
Return:If it supports judging the quality of fingerprint images, return 1; otherwise, re turn 0

9. Does the interface provide image enhancement function for stitching finge rprints

Function prototype :public static native int MOSAIC_IsSupportImageEnhance()
Parameter :null.
Return:If image enhancement function is supported, return 1; otherwise, return 0.

10. Determine whether the rolling collection function is supported

Function prototype :public static native int MOSAIC_IsSupportRollCap()
Parameter :null.
Return:If rolling collection is not supported, return 0. If only left and right rolling coll ection is supported, return 1. If only up and down rolling collection is support ed, return 2. If both left and right rolling and up and down rolling collection are supported, return 3

11. Choose splicing method

Function prototype :public static native int MOSAIC _SetRollMode(int nRollMode)
Parameter :int nRollMode Collection method: 0-plane collection; 1. Rolling collection left and right; 2 Up and down scrolling collection 3 Left and right scrolling a nd up and down scrolling collection.inputParameter .
Return:Successfully returned the collection method setting value before modification.

12. Initialize the splicing process

Function prototype :public static native int MOSAIC_Start (unsigned char *pFingerB uf,int nWidth,intnHeight)
Parameter :unsigncd char*pFingerBuf Pointing to concatenating data memory blocks for external allocation, with a size of nWidthXnHeight.inputParameter /OutPara meter .
int nWidth The width of the concatenated image should be the same as the wi dth of the captured image.inputParameter .
int nHeightThe height of the stitched image should be the same as the height of the collected image.inputParameter
Return:Call successful, return 1

13. Splicing process

Function prototype :int _stdcall MOSAIC_DoMosaic(unsigned char *pDataBuf,int nWi dth,int nHeight)
Parameter :unsigned char *pDataBuf Pointing to the memory block for data collectio n, with a size of nWidth XnHeight.inputParameter .
int nWidthThe width of the collected image should be the same as the width of the stitched image.inputParameter .

int nHeightThe height of the collected image should be the same as the hei ght of the stitched image.inputParameter .

Return:Call successful, return 1;If it returns 0, it means that the data contained in pDataBuf is not fingerprint. This stitching is null, and the stitching data still maintains the stitching result of the previous image. If it returns 0, the stitchi ng should be stopped and the stitching is completed.

14. End splicing

Function prototype :int _ stdcall MOSAIC_Stop()

Parameter :null.

Return:Call successful, return 1

15. Judging image quality

Function prototype ： public static native int MOSAIC_ImageQuality(unsigned char*pD ataBuf,int nWidth,int nHeight)

Parameter :unsigned char*pDataBuf Point to the memory block that stores the colle cted data.inputParameter .

int nWidth The width of the image.inputParameter .

int nHeight The height of the image.inputParameter .

Return:Call successful, return image quality, quality between [0100], where the high er the value, the better the quality

16. Judging fingerprint quality

Function prototype :public static native int MOSAIC_FingerQuality(unsigned char *pD ataBuf,int nWidth,int nHeight)

Parameter ： unsigned char*pDataBufPoint to the memory block that stores the colle cted data, which is a fingerprint.input Parameter .

int nWidth The width of the image.inputParameter .

int nHeight The height of the image.inputParameter .

Return:Call successful, return the quality of the fingerprint, with a quality between [0100]

17. Enhance images

Function prototype :public static native int MOSAIC_ImageEnhance(unsigned char *p SrcImg,int nWidth,int nHeight,unsigned char*pTargetImg)

Parameter ： unsigned charpSrcImg Pointing to the memory block storing fingerprint data that needs to be enhanced.inputParameter .

int nWidth The width of the image.inputParameter .

int nHeight The height of the image.inputParameter .

unsigned char*pTargetImgEnhanced fingerprint data. Memory allocated by the caller, with a size of nWidthXnHeight.OutParameter .

Return:Call successful, return 1.

18. Determine if the image is a fingerprint

Function prototype :public static native int MOSAIC_IsFinger(unsigned char *pDataBuf,int nWidth,int nHeight)

Parameter :unsigned char*pDataBufPoint to the memory block that stores the collected data, which is a single image data.inputParameter .

int nWidth The width of the image.inputParameter .

int nHeight The height of the image.inputParameter .

Return:If it is a fingerprint, return 1 instead of 0.

19. Obtain splicing interface error information

Function prototype :public static native int MOSAIC_GetErrorInfo(int nErrorNo,char pszErrorIno[256])

Parameter :char pszErrorInfo[256] A memory block used to store error messages.OutParameter .

nErrorNo error code(<0).inputParameter .

Return:If it is a valid error code, return 1, and the pszErrInfo contains incorrect information

Explanation: Obtain splicing interface error information. pszErrorInfo. The encoding characters for the information are specified in GB2312. If there are no specified characters in GB 2312, the characters specified in GB 18030 are used. The information is a string that ends with a value of 0.

20. Obtain the version of the interface

Function prototype :public static native int MOSAIC_GetVersion()

Parameter :null.

Return:Obtain the version of the interface specification. The current version is 1.00, with a return value of 100

21. Instructions for obtaining splicing interfaces

Function prototype :public static native int MOSAIC_GetDesc(char pszDesc[1024])

Parameter :pszDesc Instructions for storing interfaces.OutParameter .

Return: return 1.

Explanation: Obtain a description of the splicing interface and be able to call. pszDesc from the beginning. The encoding characters should use the characters specified in GB 2312. If there are no specified characters in GB 2312, the characters specified in GB 18030 should be used. The information is a string ending in a value of 0

22. Image stitching interface error code

The error codes for the image stitching interface refer to Table 2.1.

Table 2.1 Error code table

Error code	
-1	Parameter error. There is an error in the parameters of the given function

-2	Memory allocation failed. Not enough memory allocated
-3	Function not implemented. The function calling function has not been implemented
-6	Illegal error number
-7	No authorization
-8	Splicing not initialized

III、libFpStdAPI.so Interface Description

1. Initialization algorithm

Function prototype :public native long OpenDevice();
Parameter ： null.
Return:Call successful, return device handle; otherwise, return 0

2. Release algorithm

Function prototype :public int CloseDevice(long device);
Parameter ： long device device handle.inputParameter .
Return:Call successful, return 1，Otherwise, return 0

3. Get Image Quality

Function prototype :public native int GetImageQuality(long device,byte[] image);
Parameter ： long device device handle.inputParameter .
byte[] image Fingerprint image data，Only images with a size of 256 * 360 bytes can be allocated.inputParameter .
Return:Call successful, return score;

4. Get Image NFIQ Quality

Function prototype :public native int GetNFIQuality(long device,byte[] image);
Parameter ： long device device handle.inputParameter .
byte[] image Fingerprint image data，Only images with a size of 256 * 360 bytes can be allocated.inputParameter .
Return:Call successful, return NFIQ;

5. Comparing fingerprint features

Function prototype :public native int ZAZ_FpStdLib_CompareTemplates(int device, byte[] sTemplate, byte[] fTemplate);
Parameter ： long device device handle.inputParameter .
byte[] sTemplate Fingerprint features.inputParameter .
byte[] fTemplate Fingerprint features.inputParameter .
Return:Successfully called, returns the similarity score of two fingerprint features, with a score range of [0,100]

6. Generate fingerprint ANSI features

Function prototype :public native int CreateANSITemplate(long device,byte[] image, byte[] itemplate);
Parameter ： long device device handle.inputParameter .
byte[] image Fingerprint image data，Only images with a size of 256 * 360 bytes can be allocated.inputParameter .
byte[] itemplate Fingerprint features，Allocate a size of 1024 bytes before inp

ut.OutParameter .
Return:Call successful, return fingerprint feature size，Otherwise, return 0

7. Generate fingerprint ISO features

Function prototype :public native int CreateISOTemplate(long device,byte[] image, byte[] itemplate);
Parameter ：long device device handle.inputParameter .
byte[] image Fingerprint image data，Only images with a size of 256 * 360 bytes can be allocated.inputParameter .
byte[] itemplate Fingerprint features，Allocate a size of 1024 bytes before input. OutParameter .
Return:Call successful, return fingerprint feature size，Otherwise, return 0

8. CompressToWSQImage

Function prototype :public native int CompressToWSQImage (long device,byte[] rawImage, int w,int h,byte[] wsqImage);//
Parameter ：long device device handle.inputParameter .
byte[] rawImage Fingerprint image data.inputParameter.
int w Image width.inputParameter.
Int h Image height.inputParameter.
byte[] wsqImage : the Compress image data.
Return:Call successful, return freturn size of image compressed by WSQ.

9. UnCompressFromWSQImage

Function prototype :public native int UnCompressFromWSQImage (long device,byte[] wsqImage, int wsqSize, byte[] rawImage,int[] w,int[] h);
Parameter ：long device device handle.inputParameter .
byte[] wsqImage the Compress image data.inputParameter .
int wsqSize Length of compressed image data.inputParameter.
byte[] rawImage Decompressed image data.OutParameter .
int[] w Width of the decompressed image.OutParameter .
int[] h: Length of the decompressed image.OutParameter .
Return:Call successful, return size of uncompressed image
.